

Local Variables and Scope of Variables



- Produced Ms. Mairead Meagher,
 - by: Ms. Siobhán Roche.

Topics list

1. Local variables.

2. Scope of variables.

To think about

- How could we write a method to 'refund' an excess balance?
 - We need to return the value in balance.
 - We need to set the balance to zero.
- How can we do the first without losing the value in balance?

Unsuccessful attempt

```
public int refundBalance()  
{  
    // Return the amount left.  
    return balance;  
    // Clear the balance.  
    balance = 0;  
}
```

It looks logical, but the language does not allow it.

A method will terminate immediately after executing a return statement!

Local variables

- Methods can define their own, *local* variables:
 - Short lived, like parameters.
 - The method sets their values – unlike parameters, they do not receive external values.
 - Used for ‘temporary’ calculation and storage.
 - They exist only as long as the method is being executed.
 - They are only accessible from within the method.
 - They are defined within a particular *scope*.

Local variables

A local variable



No visibility
modifier

```
public int refundBalance()  
{  
    int amountToRefund;  
    amountToRefund = balance;  
    balance = 0;  
    return amountToRefund;  
}
```

Topics list

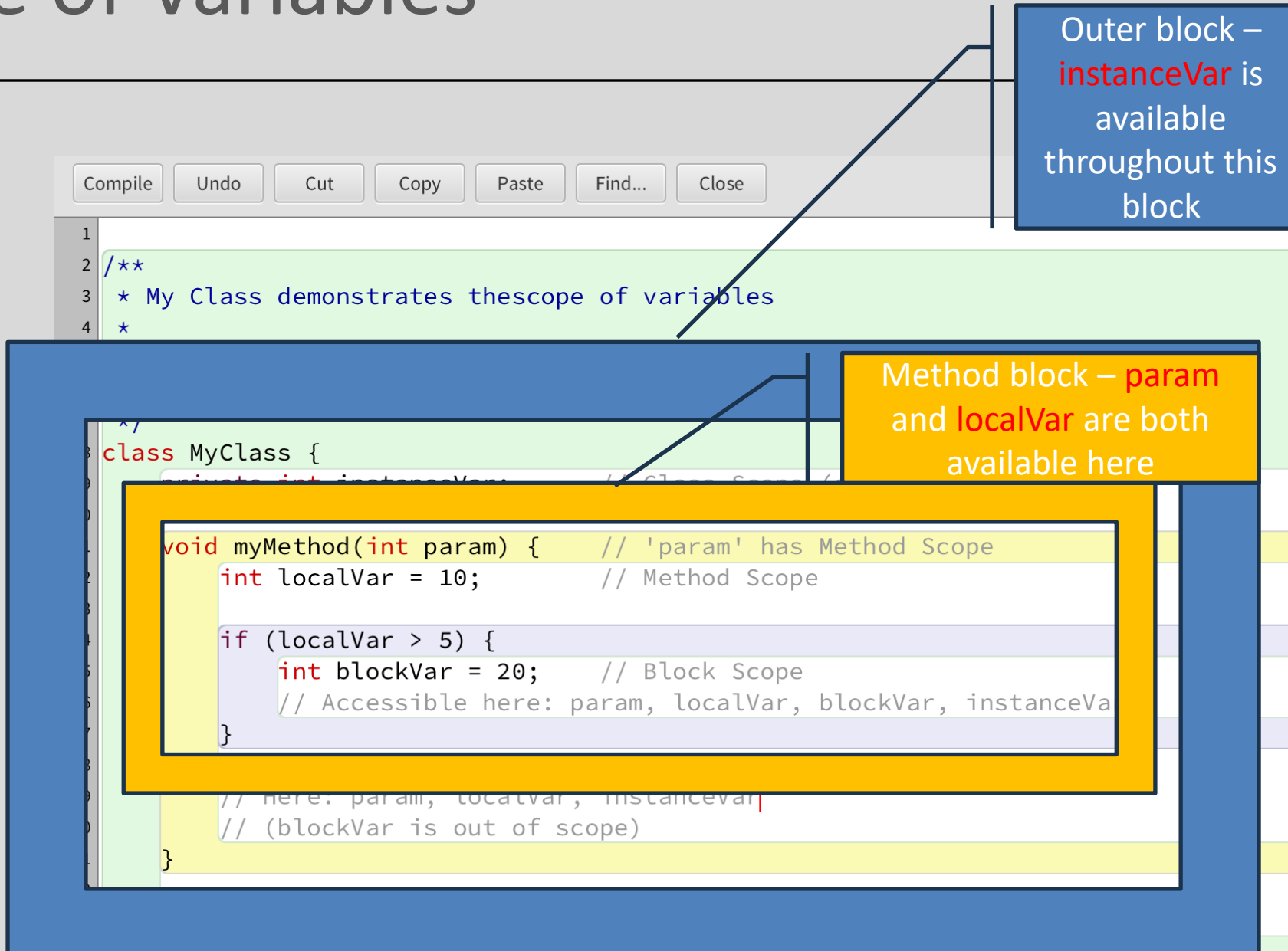
1. Local variables.

2. Scope of variables.

Scope of Variables

```
1 2 /**
3  * My Class demonstrates the scope of variables
4  *
5  * @author Mairead & Siobhan
6  * @version 25-09-2025
7  */
8 class MyClass {
9     private int instanceVar;    // Class Scope (object-level)
10
11     void myMethod(int param) {  // 'param' has Method Scope
12         int localVar = 10;     // Method Scope
13
14         if (localVar > 5) {
15             int blockVar = 20;  // Block Scope
16             // Accessible here: param, localVar, blockVar, instanceVar
17         }
18
19         // Here: param, localVar, instanceVar
20         // (blockVar is out of scope)
21     }
22
23     // Here: instanceVar
24 }
25
```


Scope of Variables



Scope and lifetime

- Each block defines a new scope.
 - Class, method and statement.
- Scopes may be nested:
 - statement block inside another block inside a method body inside a class body.
- Scope is static (textual).
- Lifetime is dynamic (runtime).

Scope and lifetime

- The scope of a field is its whole class.
- The lifetime of a field is the lifetime of its containing object.
- The scope of a local variable is the block in which it is declared.
- The lifetime of a local variable is the time of execution of the block in which it is declared.

Questions?

